



Managed by IHRD, a Govt. of Kerala undertaking

COLLEGE OF ENGINEERING
POONJAR

GROUP 1

PROJECT

SYNAPSE

PREPARED BY

Ihsana Ilyas

Albert Jossy

Abhishek TP

Adinan Azeez KT



SYSTEM ARCHITECTURE

CONTENTS

- Introduction..... 2
- System Overview..... 3
- Architectural Goals..... 4
- Software Architecture..... 5
- Layer Description..... 6
- Component Interaction.....7
- Machine Learning Overview..... 8
- Database Architecture.....10
- External Interfaces.....10
- Deployment Architecture.....11
- Security and Performance Considerations.....11
- Future Enhancements.....12
- Conclusion.....13

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Architecture Document (SAD) is to describe the overall software architecture of Synapse, an AI-powered Train Delay Prediction System that utilizes Machine Learning techniques to predict train delays before a journey begins. This document presents the architectural design, major software components, communication mechanisms, and technology stack adopted for the development of the system. It provides a clear understanding of how different layers of the application interact to deliver accurate train delay predictions.

The Software Architecture Document serves as a technical reference for developers, project supervisors, and other stakeholders involved in the project. It establishes a common understanding of the system's structure and supports the implementation, testing, maintenance, and future enhancement of the application. By documenting the architectural decisions and design principles, the SAD also ensures consistency throughout the software development life cycle.

1.2 Scope

Synapse is an intelligent web-based application designed to predict train delays for selected Indian trains before the journey commences. Unlike conventional railway applications that primarily display the current running status of trains, the proposed system estimates the expected delay at each station by analyzing historical railway data together with contextual information such as weather conditions, day of the week, weekends, seasonal travel patterns, and previous delay trends.

The initial implementation focuses on a limited number of trains to enable the development of an accurate and reliable prediction model using high-quality historical data. The architecture has been designed with scalability in mind so that additional trains, datasets, and prediction models can be incorporated in future versions without requiring significant architectural changes.

The system also supports periodic retraining of the machine learning model using newly collected operational data. This enables the prediction model to continuously improve its accuracy and adapt to changing railway conditions over time.

1.3 Intended Audience

This document is intended for individuals involved in the design, development, evaluation, and maintenance of the Synapse project. It provides sufficient architectural information for understanding the responsibilities of each software component and their interactions within the system.

The primary audience includes:

- Project Supervisor
- Software Developers
- Machine Learning Engineers
- Database Designers
- Frontend Developers
- Backend Developers
- Project Evaluators
- Future Development Teams

Each stakeholder can use this document as a reference during different phases of the software development process, including implementation, testing, deployment, and future system enhancements.

1.4 Definitions, Acronyms and Abbreviations

Term Descriptions:

- AI Artificial Intelligence
- ML Machine Learning
- SAD Software Architecture Document
- API Application Programming Interface
- REST API Representational State Transfer Application Programming Interface used for communication between frontend and backend components
- HTTP Hypertext Transfer Protocol
- JSON JavaScript Object Notation used for data exchange
- UI User Interface
- DBMS Database Management System

PostgreSQL Open-source relational database management system used for storing historical train and prediction data

React JavaScript library used for developing the frontend user interface

FastAPI Python-based backend framework used to implement RESTful APIs

Feature Engineering The process of transforming raw data into meaningful features suitable for machine learning models

Prediction Engine Software component responsible for generating train delay predictions using the trained machine learning model

1.5 References

The design and development of Synapse are based on established software engineering principles, machine learning frameworks, and publicly available technical documentation. The following references have been considered during the preparation of this document:

- IEEE Standard 42010 – Systems and Software Engineering: Architecture Description.
- Scikit-learn Documentation.
- XGBoost Documentation.
- FastAPI Documentation.
- PostgreSQL Documentation.
- OpenWeather API Documentation.
- Project Proposal Report – Synapse: AI-Powered Train Delay Prediction System using Machine Learning

2.SYSTEM OVERVIEW

2.1 Introduction

Railway transportation is one of the most widely used modes of travel in India. Every day, millions of passengers depend on trains for commuting, business, education, and tourism. However, train delays remain a significant challenge due to various operational and environmental factors. Unexpected delays often affect passengers' travel plans, connecting transportation, and scheduled activities. Although existing railway applications provide information about the current running status of trains, they generally do not predict future delays before the journey begins.

Synapse is an AI-powered Train Delay Prediction System developed to address this limitation. The system applies machine learning techniques to analyze historical train performance data together with contextual information such as weather conditions and temporal patterns to estimate the expected delay at each station along a selected route. The predicted results enable passengers to make informed travel decisions before starting their journey.

2.2 System Description

Synapse is designed as a web-based application that combines machine learning with software engineering principles to provide accurate delay predictions for selected Indian trains. The system accepts journey information from the user, processes the request through multiple software layers, and generates station-wise delay predictions along with confidence scores and the major factors influencing the prediction.

The proposed system currently focuses on a limited number of trains to ensure the availability of high-quality historical data and improve the reliability of the prediction model. This approach also simplifies the data collection and model training process during the initial development phase. The architecture, however, has been designed to support future expansion to additional trains and routes without requiring major structural changes.

Unlike conventional train tracking systems that estimate delays using only the current running status of a train, Synapse predicts delays before the journey begins by learning patterns from historical operational data. The machine learning model identifies relationships between multiple influencing factors and uses these relationships to estimate future delays at different stations.

2.3 Objectives of the System

The primary objective of Synapse is to develop an intelligent prediction system capable of estimating train delays before a train reaches each station. To achieve this objective, the system has been designed with the following goals:

- Predict expected delays at upcoming stations before the journey begins.
- Learn historical delay patterns using supervised machine learning techniques.

Improve prediction accuracy through periodic model retraining.
Provide confidence scores for every prediction.
Identify and display the major factors contributing to the predicted delay.
Provide a modular and scalable software architecture that supports future enhancements.

2.4 Major Functionalities

The proposed system performs several important functions throughout the prediction process. Initially, users enter journey details including the train number, journey date, boarding station, and destination station. The application validates the input before forwarding it to the backend server.

The backend retrieves the required contextual information and prepares the input features for the machine learning model. The trained prediction model estimates the delay expected at each station and calculates the confidence associated with the prediction. Finally, the system presents the predicted delays, confidence values, and major contributing factors through an interactive web interface.

2.5 Overall Workflow

The overall workflow of Synapse consists of the following stages:

1. The user enters journey details through the web application.
2. The frontend sends the request to the backend using REST APIs.
3. The backend validates the received information.
4. The Business Logic Layer prepares the input features required for prediction.
5. Historical train data and weather information are collected.
6. The Machine Learning Layer loads the trained prediction model.
7. The model predicts the expected delay for each station.
8. The prediction results are stored in the database for future analysis.
9. The predicted delay, confidence score, and major contributing factors are displayed to the user.

This workflow ensures that every prediction is generated through a systematic process involving data preprocessing, machine learning inference, and result present.

3. ARCHITECTURAL GOALS

The software architecture of Synapse has been designed to satisfy both the functional and non-functional requirements of the proposed system. Since the application integrates web technologies, machine learning models, and databases, the architecture must ensure that each component performs its responsibilities independently while maintaining effective communication with other components.

The major architectural goals considered during the design process are discussed below.

3.1 Modularity

The system follows a layered architecture in which each layer performs a specific responsibility. Separating the application into independent modules simplifies development, testing, debugging, and future maintenance. Changes made within one module have minimal impact on the remaining components of the system.

3.2 Scalability

The architecture has been designed to accommodate future expansion. Although the current implementation focuses on a limited number of trains, additional routes, datasets, and prediction models can be incorporated without major modifications to the existing software architecture.

3.3 Maintainability

Maintainability is one of the primary objectives of the system architecture. Individual software components have clearly defined responsibilities, allowing developers to modify or improve specific modules without affecting the entire application. This approach reduces maintenance effort and simplifies future upgrades.

3.4 Reliability

Reliable prediction results are essential for assisting passengers in planning their journeys. The system therefore emphasizes consistent communication between software layers, proper validation of user inputs, and dependable storage

of historical train data. The machine learning model is also periodically retrained to maintain prediction accuracy as new operational data becomes available.

3.5 Performance

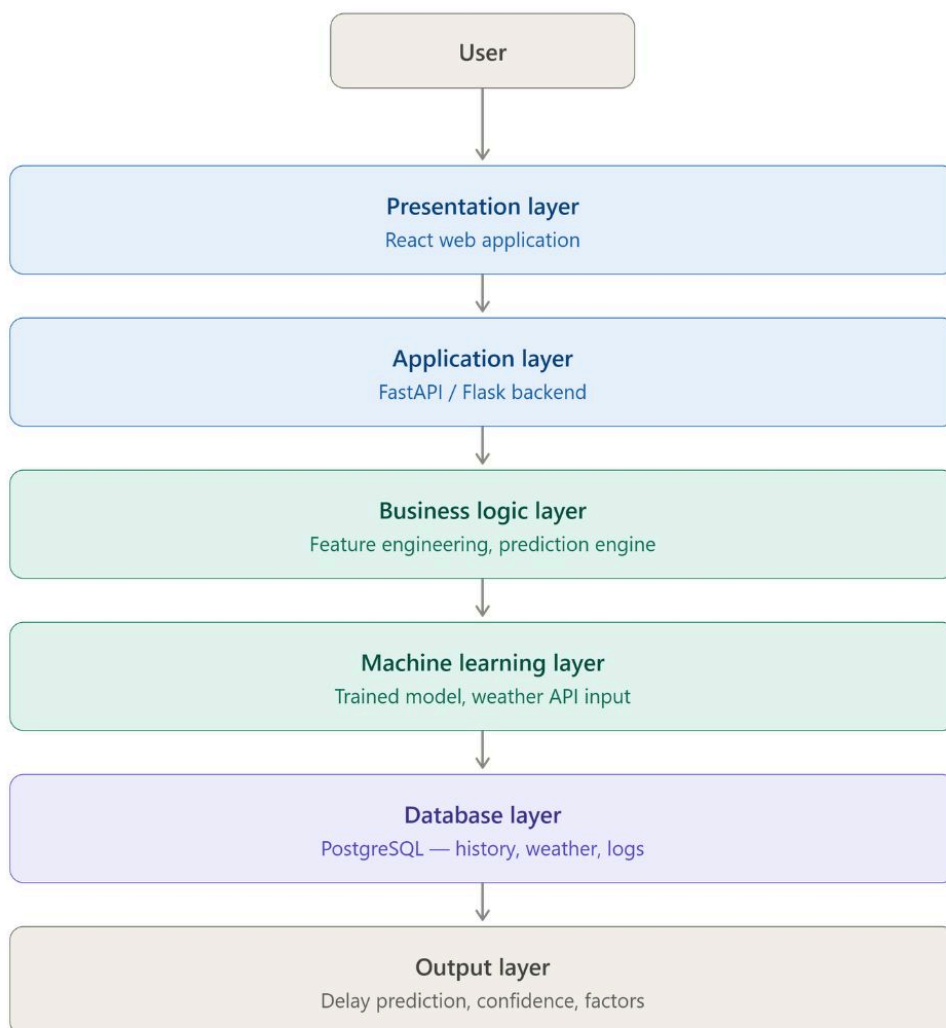
The prediction system should generate results within a reasonable time after receiving user input. Efficient communication between the frontend, backend, database, and machine learning components minimizes processing delays and provides a responsive user experience.

3.6 Flexibility

The proposed architecture supports the integration of additional datasets, external APIs, and improved machine learning algorithms in future versions of the system. This flexibility allows Synapse to evolve without requiring a complete redesign of the software architecture.

4. SOFTWARE ARCHITECTURE

Synapse follows a Layered Software Architecture, in which the application is divided into multiple logical layers. Each layer is responsible for a specific set of operations and communicates only with adjacent layers. This design promotes modularity, maintainability, and ease of development by separating user interface components from business logic, machine learning processing, and database management.



The architecture consists of **six** major layers:

1. Presentation Layer
2. Application Layer
3. Business Logic Layer
4. Machine Learning Layer
5. Database Layer
6. Output Layer

Communication between these layers occurs primarily through RESTful APIs and internal software interfaces.

4.2 Architectural Style

The system adopts the Layered Architecture Pattern, which organizes software components into independent layers with clearly defined responsibilities. Each layer provides services to the layer above it while utilizing services from the layer below it.

This architectural style was selected because it offers several advantages for machine learning applications. It enables independent development of the frontend, backend, prediction engine, and database components. The separation of concerns also improves software maintainability, testing, and future scalability.

4.3 Architecture Components

The Presentation Layer provides the graphical interface through which users interact with the application. It collects journey information and displays prediction results.

The Application Layer manages incoming requests from the frontend, validates user inputs, and routes requests to the appropriate backend services.

The Business Logic Layer performs feature engineering, data preparation, and coordinates communication with the machine learning model.

The Machine Learning Layer contains the trained prediction model and associated training modules. It generates delay predictions and supports periodic retraining using newly collected data.

The Database Layer stores historical train records, weather data, prediction logs, and model metadata required for system operation.

Finally, the Output Layer formats the prediction results and presents them to the end user through the web interface.

4.4 Advantages of the Selected Architecture

The layered architecture offers several advantages for Synapse. It simplifies software maintenance by isolating individual components and reducing dependencies between modules. The architecture also supports scalability by allowing new features and services to be integrated with minimal modifications to existing components.

Furthermore, the architecture enables efficient collaboration among development teams, as frontend, backend, database, and machine learning components can be developed independently while following well-defined communication interfaces.

5. LAYER DESCRIPTION

The software architecture of Synapse is organized into six logical layers. Each layer performs a specific responsibility and collaborates with adjacent layers to process user requests and generate train delay predictions. This separation of responsibilities improves software maintainability, modularity, and scalability.

5.1 Presentation Layer

The Presentation Layer serves as the interface between users and the Synapse system. It provides a responsive web application through which passengers can enter journey information and view prediction results. The frontend is planned to be developed using React, HTML, CSS, and JavaScript to provide a modern and interactive user experience.

This layer is responsible for collecting user inputs such as train number, journey date, boarding station, and destination station. It performs basic client-side validation before forwarding the request to the backend through RESTful APIs. Once the prediction process is completed, the Presentation Layer displays the predicted delay, confidence score, and major contributing factors in a clear and user-friendly format.

5.2 Application Layer

The Application Layer acts as the communication bridge between the frontend and the core processing modules. It receives requests from the Presentation Layer, validates the incoming data, manages API routing, and coordinates communication with the Business Logic Layer.

The backend of Synapse is proposed to be implemented using FastAPI because of its high performance, lightweight architecture, and seamless integration with Python-based machine learning libraries. This layer ensures that only valid requests are processed and appropriate responses are returned to the user.

5.3 Business Logic Layer

The Business Logic Layer contains the core operational logic of the application. It transforms user requests into a format suitable for the machine learning model by performing feature engineering and data preprocessing. This includes encoding categorical variables, generating derived features, and preparing the input dataset required by the prediction engine.

After preparing the features, this layer invokes the trained machine learning model and processes the prediction results before forwarding them to the output module. It also coordinates communication between the backend, database, and machine learning components.

5.4 Machine Learning Layer

The Machine Learning Layer is responsible for training, managing, and executing the prediction model. Historical train data is used to train supervised machine learning algorithms capable of estimating delays for selected trains.

During prediction, the trained model receives processed feature vectors from the Business Logic Layer and estimates the expected delay for each station. The layer also calculates confidence scores associated with the predictions. To improve long-term performance, the architecture supports periodic retraining using newly collected train data, allowing the model to adapt to changing operational conditions.

5.5 Database Layer

The Database Layer manages the storage and retrieval of information required by the prediction system. PostgreSQL has been selected as the primary database due to its reliability, scalability, and strong support for structured data management.

The database stores historical train records, newly collected operational data, historical weather information, prediction logs, and metadata related to the trained machine learning models. By maintaining these datasets, the system supports both real-time prediction and future model retraining.

5.6 Output Layer

The Output Layer is responsible for presenting the final prediction results to the user. After receiving processed information from the Business Logic Layer, it formats the prediction output into an easily understandable form.

The displayed information includes the predicted delay at each station, confidence percentage, and the major factors influencing the prediction. The Output Layer ensures that complex machine learning results are translated into meaningful information that assists passengers in planning their journeys effectively.

6. COMPONENT INTERACTION

6.1 Overview

The components of Synapse communicate through a layered architecture in which each component performs a specific function before forwarding the request to the next layer. This approach ensures that the responsibilities of data collection, processing, prediction, storage, and presentation remain independent while maintaining smooth communication between modules.

The interaction begins when the user submits journey information through the web interface. The request is processed by the backend server, where the required features are generated before being forwarded to the trained machine learning model. The prediction generated by the model is then formatted and returned to the user.

6.2 Component Interaction Flow

The interaction between system components follows the sequence shown below.

- The user enters the journey details through the web interface.
- The Presentation Layer sends a REST API request to the backend server.
- The Application Layer validates the received request.
- The Business Logic Layer prepares the required input features.
- Historical train data and weather information are retrieved from the database.
- The Machine Learning Layer loads the trained model.
- The model predicts station-wise delays.
- Prediction results are stored in the Prediction Log.
- The processed response is returned to the Presentation Layer.
- The predicted delays and confidence scores are displayed to the user.

6.3 Communication Between Components

Source Component	Destination Component	Communication Method
Presentation Layer	Application Layer	REST API
Application Layer	Business Logic Layer	Internal Function Calls
Business Logic Layer	Database Layer	SQL Queries
Business Logic Layer	Machine Learning Layer	Python Function Calls
Machine Learning Layer	Business Logic Layer	Prediction Response
Business Logic Layer	Presentation Layer	REST API Response

7. MACHINE LEARNING OVERVIEW

7.1 Overview

Machine Learning forms the core of the Synapse prediction system. Instead of relying solely on the current running status of trains, the system learns historical delay patterns and estimates future delays using supervised learning techniques. The machine learning architecture consists of data collection, preprocessing, feature engineering, model training, evaluation, prediction, and continuous learning.

7.2 Dataset

The prediction model is trained using historical railway data collected for selected trains. The dataset contains information related to train operations, temporal factors, and weather conditions that influence train delays.

The primary attributes include:

- Train Number
- Train Name
- Station
- Scheduled Arrival Time
- Actual Arrival Time
- Delay (Minutes)
- Journey Date
- Day of the Week
- Month
- Weather Condition
- Rainfall
- Weekend Indicator
- Previous Station Delay

7.3 Data Preprocessing

Raw datasets often contain missing values, duplicate records, and inconsistent information. Before training the machine learning model, the collected data undergoes preprocessing to improve its quality and suitability for prediction.

The preprocessing stage includes:

- Data Cleaning
- Missing Value Handling
- Duplicate Removal
- Categorical Data Encoding
- Feature Selection
- Data Normalization (where required)

7.4 Model Training

The processed dataset is divided into training and testing datasets. Multiple supervised machine learning algorithms are evaluated, and the model with the highest prediction accuracy is selected for deployment.

The algorithms considered include:

1. Random Forest Regressor
2. XGBoost
3. Gradient Boosting
4. LightGBM

7.5 Prediction Process

During prediction, the trained model receives processed input features prepared by the Business Logic Layer. The model estimates the expected delay at each station and generates confidence values associated with each prediction.

The prediction results are then forwarded to the Output Layer for presentation.

7.6 Continuous Learning

To improve long-term prediction performance, Synapse supports periodic retraining of the machine learning model. Newly collected train records are added to the historical dataset, allowing the model to adapt to changing railway conditions and improve prediction accuracy over time.

8. DATABASE ARCHITECTURE

8.1 Overview

The Database Layer is responsible for storing and managing all persistent information required by the Synapse system. It maintains historical train records, weather information, prediction logs, and model metadata that support both prediction and future retraining.

PostgreSQL has been selected because of its scalability, reliability, and strong support for relational data.

8.2 Database Components

1. The database contains several logical entities that store different categories of information.
2. Historical Train Data.
3. Stores operational records collected for selected trains, including scheduled and actual arrival times, station details, and recorded delays.
4. Weather Data
5. Maintains historical weather information collected for corresponding journey dates and locations. These records are used during feature engineering and model training.
6. Prediction Logs
7. Stores previously generated predictions, confidence scores, and prediction timestamps. These records assist in evaluating model performance.
8. Model Metadata
9. Stores information about trained machine learning models, including model version, training date, evaluation metrics, and deployment status.

8.3 Database Operations

The primary database operations performed by Synapse include:

- Reading historical train records
- Retrieving weather information
- Storing prediction logs
- Updating newly collected datasets
- Retrieving model metadata

9. EXTERNAL INTERFACES

9.1 Overview

Synapse interacts with external services to obtain additional information required during the prediction process. These external interfaces provide contextual data that improves prediction accuracy without requiring manual input from users.

9.2 Weather API

Weather conditions significantly influence train operations. The system retrieves weather information from a Weather API during prediction to obtain parameters such as rainfall, temperature, humidity, and weather conditions for the selected journey date.

This information is incorporated into the feature engineering process before generating predictions.

9.3 REST API Communication

Communication between the frontend application and backend server is performed using RESTful APIs. User requests are transmitted as HTTP requests containing JSON-formatted data, while prediction results are returned using the same communication mechanism.

REST APIs provide a standardized interface that simplifies communication between different software components and supports future integration with mobile applications.

10. DEPLOYMENT ARCHITECTURE

10.1 Overview

The deployment architecture describes how the various software components of Synapse are deployed and interact during execution. The system follows a client-server architecture where the frontend application, backend services, machine learning model, and database operate as separate but interconnected components.

10.2 Deployment Components

The frontend application is hosted on a web server and accessed through a standard web browser. The backend server hosts the REST APIs and business logic developed using FastAPI. The trained machine learning model resides within the backend environment and is loaded whenever a prediction request is received.

The PostgreSQL database stores historical train data, weather information, prediction logs, and model metadata. Communication between the backend server and the database is achieved through secure database connections.

10.3 Deployment Workflow

The deployment process follows the sequence below:

1. The user accesses the Synapse web application through a browser.
2. Journey details are submitted to the backend server.
3. The backend processes the request and communicates with the database.
4. The Machine Learning Layer generates train delay predictions.
5. The prediction results are returned to the frontend and displayed to the user.

The modular deployment architecture allows each component to be maintained, updated, and scaled independently, ensuring flexibility and improved system performance.

11. SECURITY AND PERFORMANCE CONSIDERATIONS

11.1 Overview

Security and performance are important quality attributes of the Synapse system. Since the application processes user inputs, historical datasets, and machine learning models, appropriate measures must be taken to ensure reliable operation, secure communication, and efficient prediction generation. Although Synapse is primarily developed as an academic project, the architecture incorporates several practices that can be extended for real-world deployment.

11.2 Security Considerations

- The system validates all user inputs before processing prediction requests. Input validation helps prevent invalid or incomplete data from reaching the prediction engine and reduces the possibility of unexpected system behavior.
- Communication between the frontend and backend is carried out using RESTful APIs. During deployment, these APIs can be secured using HTTPS to encrypt data exchanged between the client and the server. This prevents unauthorized interception of information during transmission.
- The backend is responsible for validating incoming API requests before forwarding them to the Business Logic Layer. Error handling mechanisms are incorporated to ensure that invalid requests generate appropriate error responses rather than causing system failures.
- The database is accessible only through the backend server, preventing direct access from client applications. This architecture helps protect historical train data, prediction logs, and model-related information from unauthorized modifications.
- Authentication and user management have been kept optional in the current version of the project. However, the modular architecture allows authentication mechanisms such as JWT (JSON Web Token) or OAuth to be integrated in future versions if personalized services are introduced.

11.3 Performance Considerations

1. The performance of Synapse depends on efficient communication between the frontend, backend, database, and machine learning components. The system is designed to minimize response time while maintaining prediction accuracy.
2. Historical datasets required for prediction are stored in a structured database, allowing faster retrieval during feature preparation. The trained machine learning model is loaded into memory during application execution to reduce repeated loading overhead and improve prediction speed.
3. FastAPI has been selected as the backend framework because it supports asynchronous request handling and provides high performance for Python applications. The layered architecture also enables independent optimization of individual modules without affecting the remaining components.
4. Future versions of the system may incorporate caching mechanisms, optimized database indexing, and cloud-based deployment to further improve performance when supporting larger datasets and a greater number of users.

11.4 Reliability and Maintainability

1. The layered architecture adopted by Synapse contributes significantly to system reliability and maintainability. Each layer performs a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces. This separation simplifies debugging, testing, and future enhancements.
2. Error handling mechanisms are implemented at different stages of the application to ensure that failures in one component do not affect the entire system. Logging of prediction requests and system events further assists developers in identifying and resolving issues during maintenance.
3. The modular design also allows new machine learning algorithms, additional datasets, or external APIs to be integrated with minimal changes to the existing architecture.

12. FUTURE ENHANCEMENTS

- Although Synapse is initially designed for predicting delays of selected Indian trains, the proposed architecture provides a strong foundation for future expansion. The modular design enables additional functionalities and services to be incorporated without significant architectural modifications.
- One of the primary future enhancements is the expansion of the prediction model to support a larger number of trains across the Indian Railway network. As more historical datasets become available, the system can be trained using diverse operational conditions, improving both prediction accuracy and geographical coverage.
- Integration with official railway APIs can further enhance the capabilities of Synapse by providing real-time train information. Combining live operational data with historical patterns would allow the prediction engine to produce more accurate and dynamic delay estimates.
- The current implementation utilizes weather information as one of the major influencing factors. Future versions may integrate additional contextual information such as maintenance schedules, temporary speed restrictions, infrastructure conditions, and platform availability to improve prediction quality.
- A dedicated mobile application can also be developed to provide passengers with train delay predictions directly on their smartphones. Personalized notifications, journey reminders, and real-time prediction updates could further enhance the user experience.
- Explainable Artificial Intelligence (XAI) techniques may also be incorporated in future versions to provide more detailed explanations of prediction results. Instead of simply displaying the predicted delay, the system could visually demonstrate how different factors such as rainfall, weekends, or historical delay trends contributed to the final prediction.

- Cloud deployment represents another important future enhancement. Hosting the application and machine learning services on cloud platforms would improve scalability, availability, and reliability while supporting larger datasets and increased user traffic.
- Finally, the architecture supports continuous learning through periodic retraining of the machine learning model. As additional historical train records are collected, the model can be updated regularly to adapt to changing railway conditions and improve long-term prediction performance

13. CONCLUSION

Synapse has been designed as a modular, scalable, and intelligent train delay prediction system that combines modern web technologies with machine learning techniques. The layered software architecture separates the application into independent components, allowing each layer to perform a specific responsibility while communicating efficiently with adjacent layers. This architectural approach improves maintainability, simplifies future development, and supports the integration of additional functionalities as the project evolves.

The system utilizes historical train records and contextual information, such as weather conditions and temporal factors, to estimate expected train delays before the journey begins. By employing supervised machine learning techniques and supporting periodic model retraining, Synapse aims to provide more accurate and adaptive predictions compared to conventional railway tracking systems that rely primarily on current train status.

The proposed architecture also emphasizes modularity, scalability, and flexibility, enabling future integration with official railway APIs, cloud deployment, mobile applications, and explainable artificial intelligence techniques. These enhancements can significantly improve the usability and practical applicability of the system.

Overall, the Software Architecture Document provides a comprehensive description of the architectural design and major software components of Synapse. It serves as a technical reference for developers, project supervisors, and future development teams, ensuring that the system can be implemented, maintained, and extended in a structured and consistent manner.